



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24402

学 号： 24068240216

姓 名： 李晓翠

实验（1）

【实验目的】

1. 掌握 AnythingLLM 本地服务部署、工作区参数获取及自定义环境变量配置方法。
2. 运用 Python subprocess 调用 curl 对接 AnythingLLM API 接口，解决鉴权、中文编码、环境变量读取等实际问题。
3. 基于已有代码迭代开发，掌握 Agent 工具函数开发、注册及提示词设置，实现关键词触发本地知识库查询功能。
4. 对比在线大模型与本地知识库问答效果，理解私有知识库与大模型联合应用模式。

【实验内容】

1.2 实验过程：

AnythingLLM 本地服务部署与配置

（1）启动 AnythingLLM 本地服务

安装并启动 AnythingLLM Desktop 应用，确认服务运行在 <http://localhost:3001>

在系统设置中启用 API 访问功能，生成 API Key

创建新工作区（Workspace），在设置页面获取该工作区的 Slug ID（如 my-knowledge-base）

（2）环境变量配置

在 .env 文件和 env.example 文件中新增以下配置项：

1.2 实验结果:

API 密钥	创建者	创建时间
T0T7HH8-DJ24XY2-G209DPC-632WM8N	--	2026-04-20T07:11:53.389Z

```
#ANYTHINGLLM_API_KEY
ANYTHINGLLM_API_KEY="T0T7HH8-DJ24XY2-G209DPC-632WM8N"
#ANYTHINGLLM_WORKSPACE_SLUG
ANYTHINGLLM_WORKSPACE_SLUG="workspace-1234567890"
```

Agent 工具函数开发

(1) 项目结构准备

创建 `practice04` 目录，复制 `practice03` 中的基础代码，在此基础上进行功能扩展。

(2) 实现 `anythingllm_query` 工具函数

2. 实现了 `anythingllm_query` 函数:

- 使用 `subprocess` 模块调用 `curl` 命令
- 访问 `http://localhost:3001/api/v1/workspace/{workspace_slug}/chat` API 接口
- 通过 `message` 字段发送查询
- 使用 API 密钥进行认证
- 从 `.env` 文件中读取 `ANYTHINGLLM_API_KEY` 和 `ANYTHINGLLM_WORKSPACE_SLUG` 变量
- 处理中文编码问题，使用 UTF-8 编码

3. 更新了 `tool_client.py` :

- 添加了 `anythingllm_query` 工具定义
- 更新了工具映射字典
- 更新了系统提示词，明确当用户提到"文档仓库"、"文件仓库"、"仓库"时触发 `anythingllm_query` 工具

You: 请列出我的AnythingLLM文档仓库中的所有文件

Assistant:

```
```json
{
 "tool_call": {
 "name": "anythingllm_query",
 "parameters": {
 "message": "列出文档仓库中的所有文件"
 }
 }
}
```
```

Tool Result: Error: API returned null error. Raw response: {"id": "23baec5b-db7f-41d3-ac93-4c5904f925b9", "type": "textResponse", "sources": [], "close": true, "error": null, "textResponse": "\n\n文档仓库中目前包含以下文件: \n\n1. **白鹿原介绍.txt** - 描述信息未知", "thoughts": ["@agent is executing `document-s ummarizer` tool {\n \n \"action\": \"list\"\n}", "@agent: Looking at the available docum...]

提示: 请检查 AnythingLLM 服务是否正常运行, 或查看服务器日志。

关键技术说明:

| 问题 | 解决方案 |
|--------|---|
| 中文编码乱码 | 使用 <code>ensure_ascii=False</code> 和 <code>charset=utf-8</code> 请求头 |
| API 鉴权 | 通过 <code>Authorization: Bearer</code> 头部传递 API Key |
| 环境变量读取 | 使用 <code>python-dotenv</code> 库加载 <code>.env</code> 文件 |
| 响应解析 | 解析 JSON 响应, 提取 <code>textResponse</code> 字段 |

完成了开启 AnythingLLM 的本地服务。创建 API KEY。在新建的工作区的设置页面找到向量数据库的 Slug ID; 在 `.env` 文件和 `env.example` 文件新增 `ANYTHINGLLM_API_KEY`、



ANYTHINGLLM_WORKSPACE_SLUG 变量；创建目录，实现了 anythingllm_query 函数：

使用 subprocess 模块调用 curl 命令，访问 `http://localhost:3001/api/v1/workspace/{workspace_slug}/chat` API 接口，通过 message 字段发送查询，使用 API 密钥进行认证从 .env 文件中读取 ANYTHINGLLM_API_KEY 和 ANYTHINGLLM_WORKSPACE_SLUG 变量，处理中文编码问题，使用 UTF-8 编码

更新了 tool_client.py：

添加了 anythingllm_query 工具定义，更新了工具映射字典，更新了系统提示词，明确当用户提到"文档仓库"、"文件仓库"、"仓库"时触发 anythingllm_query 工具，更新了 process_tool_call 函数以支持新工具

更新了 env.example：

添加了 ANYTHINGLLM_API_KEY 配置项，添加了 ANYTHINGLLM_WORKSPACE_SLUG 配置项

创建了 chat_client.py：

添加了 anythingllm_query 工具功能，支持流式响应和工具调用新功能

环境配置验证：

AnythingLLM 本地服务成功启动，API 端口 3001 可访问

API Key 和工作区 Slug 正确配置，环境变量读取正常

在线模型 vs 本地知识库对比：

| 对比维度 | 在线大模型（Qwen3-Coder） | 本地知识库（AnythingLLM） |
|-------------|--------------------|--------------------|
| 知识来源 | 预训练通用知识 | 用户上传的私有文档 |
| 回答时效性 | 知识截止于训练日期 | 实时基于最新上传文档 |
| 隐私性 | 数据上传至第三方 | 完全本地运行，数据不出域 |
| 回答准确性（专业领域） | 通用回答，可能不够精确 | 基于具体文档，回答精准 |
| 响应速度 | 较快（2-3 秒） | 稍慢（含检索时间） |
| 适用场景 | 通用问题、代码生成 | 企业内部文档查询、私有知识问答 |

【实验总结】

本次实验完成了 OpenRouter 在线模型 API 接入，通过配置 .env 文件实现 Qwen3-Coder 在线大模型调用，并体验了在线 LLM 的响应速度与问答能力；同时熟练完成 AnythingLLM 本地服务部署、核心参数获取及环境变量配置，基于 subprocess 编写工具函数，借助 curl 调用本地聊天接口，解决了接口鉴权、中文编码与环境变量读取等问题，遇到接口故障可查阅官方文档进行排查。随后改造 chat_client.py，注册新工具、适配函数调用并优化系统提示词，设置关键词触发机制，实现语义自动调取本地知识库查询的功能，经测试项目可正常切换在线大模型问答与私有知识库问答两种模式，还同步更新了 README 使用说明。通过本次实践，理解了 Agent 工具开发、在线大模型对接与本地知识库集成的核心逻辑，掌握了环境变量配置、子进程调用、接口鉴权、关键词路由等关键技术，为后续开

发私有知识库智能问答 Agent 积累了实践基础。

通过本次实验，完成了以下技术实践：

本地服务部署掌握了 AnythingLLM 的安装、工作区创建、API 启用和参数获取流程。接口对接使用 Python subprocess 调用 curl 实现 HTTP 请求，解决了鉴权头构造、中文 JSON 编码、响应解析等实际问题。Agent 工具开发理解了工具函数的定义规范、注册机制和提示词设计原则，实现了基于关键词的自动路由（"文档仓库" → 本地知识库查询）。混合架构设计体验了"在线大模型 + 本地知识库"的联合应用模式，在线模型负责通用理解和生成，本地知识库提供精准私有数据支持。

关键技术点：

环境变量管理（.env + python-dotenv）；子进程调用外部命令（subprocess.run）；RESTful API 鉴权与编码处理；Function Calling / Tool Use 机制；系统提示词工程（关键词触发策略）

不足与改进方向：

当前不足是使用 subprocess 调用 curl 较为繁琐，后续可改用 requests 库直接发送 HTTP 请求，代码更简洁。改进方向可为，增加错误重试机制和超时控制，实现多工作区切换支持，添加知识库搜索结果的可信度评分，优化提示词，支持更自然的语义触发（不仅依赖关键词）